From 1.144× to 1.114× Kissat: A Diagnostic Left in the Production Build, and Factor at Matched Scan Volume

Tactix development report for Andrew

4 September 2026

1 Objective and outcome

The objective has been sharpened. It is no longer only to come within 10 % of Kissat on Tier D Ranks 1–40, but to close every identified implementation gap: every case where Tactix performs the same work as Kissat and takes longer. Within 10 % is the floor, not the finish line. B28+C5 remains the regression gate and N1–N4 are preserved throughout.

Outcome. Ranks 1–40 moved from 8,781.1 s to **8,546.9 s**, which is **1.114**× Kissat's 7,675.1 s, and B28+C5 improved from 0.495 to **0.465**. Every pushed commit is 40/40 valid with 10,000 fuzz cases and zero failures on both legs. Every repair below is behaviour-preserving: the solver counters are identical between arms on every instance measured, so these are pure unit-cost recoveries and none of them changes the search.

2 The largest repair: a diagnostic that shipped

BVE ran an occurrence audit on every candidate elimination. It walked both polarities' entire ordered occurrence streams, dereferenced every large clause, ran a membership scan over each of them twice, and built a literal vector for each non-garbage one. Its only consumers are two event-trace calls, which are empty inline functions unless the tracing build option is set. In a production build all of that work was computed and discarded. Kissat's elimination has no counterpart.

Compiling it out is worth **170.6 s** across the tier, the largest single repair of the project so far. On Rank 21, BVE fell from 44.07 s to 33.82 s and the instance from 143.0 s to 131.7 s; Rank 9 fell from 106.5 s to 98.1 s.

The obvious generalisation does not hold, and it is worth recording why. Fourteen other trace calls compute a real argument on hot paths: an uncapped scan of a literal's binary prefix, or a hash lookup for a binary's clause identifier, on paths as hot as every binary addition. Guarding all of them measured nothing, twice. The compiler removes a pure inline computation whose result feeds an empty function; what it cannot remove is one that allocates, which is exactly why the BVE audit survived. The rule to apply is therefore narrow: guard the diagnostics that allocate, and leave the rest to the optimiser.

3 Factor, where the work is provably matched

Factor is the cleanest case of the sharpened objective. On Rank 22 Kissat charges 733,048,812 factor ticks and Tactix 764,237,682, a 4 % difference in the same currency, and Kissat spends 8.9 s where Tactix spent 25.2 s.

Instrumenting the charge showed 90.5 M scans covering 639.6 M streamed entries. Per entry Tactix took two random cache misses where Kissat takes one. Kissat's skip mark is a clause header bit, read from the same cache line as the size test it must perform anyway; Tactix's was an epoch array indexed

by occurrence identifier, a second random access. Moving both marks into the clause header removed one miss per entry. That then made the occurrence identifier dead in both scans, so the watch record fell from 16 bytes to 8 and a scan streams the width Kissat's does.

	Rank 22 Factor	Rank 17 Factor
baseline	25.02 s	11.60 s
marks in the clause header	22.73 s	11.14 s
two-word watch record	19.93 s	10.95 s

Tier-wide Factor fell from 281.8 s to 239.1 s. Separately, replacing the remainder pool's map of bucket vectors with an open-addressed table of identifiers removed tens of millions of allocations per application and took Rank 17 from 273.7 s to 257.4 s.

4 Two methodological corrections

The second Factor step had been refuted earlier in the same session and was right to retry. That measurement was taken while every entry still cost two cache misses, which hid the streaming cost entirely, and it paid for the smaller record with a side array. Neither confound applied once the marks moved. A null result is only as good as the conditions it was taken under.

More seriously, the early hand-written A/B campaigns ran at four CPUs and 16 GiB. That is the documented *debug* contract in the benchmark runner; the gates run at twelve CPUs and 48 GiB. Both arms shared the shape, so the signs of those results hold, but a bandwidth-sensitive measurement read at the wrong allocation is not evidence. Every campaign after that point uses the gate's shape, and the affected results were re-run.

5 Where the remaining gap is

Over the 39 ranks both campaigns report, the per-rule position is:

rule	Tactix	Kissat	gap	ratio
Substitute	287.3 s	169.6 s	+117.8	1.69
Factor	239.1 s	129.8 s	+109.4	1.84
Vivify	679.5 s	576.6 s	+102.9	1.18
Congruence	322.2 s	245.8 s	+76.4	1.31
Strengthen	112.3 s	36.1 s	+76.2	3.11
BVE	261.2 s	194.1 s	+67.1	1.35
TransitiveReduce	49.2 s	34.4 s	+14.8	1.43
Backbone	67.9 s	69.4 s	−1.6	0.98
Sweep	445.3 s	564.9 s	−119.6	0.79

The net is +443 s. Sweep is 120 s *faster* than Kissat across the tier, which corrects an earlier report section that named Sweep the largest remaining gap on the strength of a single rank.

Two of these are confirmed matched-work cases and are therefore the right next targets. Substitute substitutes 2,069,131 variables on Rank 22 against Kissat's 2,086,279; Factor's scan volume matches within 4 %. Strengthen's 3.11 is not a like-for-like comparison: Tactix checks 3.65 M clauses where Kissat checks about 26 M, and subsumes 197,796 against 105,005.

Propagation was checked and is not a general gap, although it is 42 % of the runtime. It looked 1.5× slower on Rank 31, but that instance runs 2.7× Kissat's conflicts and therefore carries longer watch

lists. On Rank 22, where both solvers perform 1.21 G propagations, the rates are 14.1 M/s and 15.4 M/s, within 9 %.

6 Limitations

Rank 31 remains the worst instance at 671 s against Kissat's 262 s, and its gap is search trajectory rather than implementation: it takes 2.7× the conflicts. That is a separate problem from the one this report addresses and it is not closed.

Two documentation budgets are over: `PROGRESS.md` is 3,773 lines against 350 and `docs/implementation-notes.md` 591 against 560. Compaction was not requested, so the debt is reported rather than paid.

Evidence for every number is in `benchmarks/reports/2026-09-02-unit-cost-and-fidelity-stack.md`, sections 3.12 to 3.18, with campaigns `5010` through `5038`.